

Hardware Heroes — Technische Dokumentation

*Interaktives Lernspiel zu PC-Grundlagen und Betriebssystem-Scheduling. Diese Dokumentation richtet sich an **Dozentinnen und Dozenten** und erklärt, **wo** was liegt, **was** die einzelnen Dateien tun und **wie** sich Inhalte anpassen lassen.*

1. Überblick

Hardware Heroes ist ein browserbasiertes Lernspiel, das PC-Hardware und einige Betriebssystem-Konzepte spielerisch vermittelt. Es besteht aus einem Startmenü und drei aufeinander aufbauenden Levels (leicht → mittel → schwer). Gespielt wird komplett im Browser – es gibt **keine Installation, keinen Build-Prozess und kein Backend**.

Zielgruppe: Schülerinnen/Schüler bzw. Studierende der Grundlagen Informationstechnik.

Lernziele:

- Level 1 – die wichtigsten PC-Bauteile kennen und erkennen (CPU, RAM, SSD, GPU, Mainboard)
- Level 2 – CPU-Scheduling verstehen (FCFS, SJF, Round-Robin, Priorität, Kontextwechsel)
- Level 3 – den Zusammenbau eines PCs nachvollziehen (Komponenten einbauen, Kabel stecken)

Live-Version: <https://luisgetlost.me> (gehostet über GitHub Pages)

2. Schnellstart

Im Browser öffnen: `index.html` per Doppelklick öffnen – das genügt, um lokal zu spielen. Für die 3D-Level (2 und 3) wird eine bestehende **Internetverbindung** benötigt, da die 3D-Bibliothek *Three.js* über ein CDN geladen wird, sowie ein Browser mit **WebGL** (alle aktuellen Browser: Chrome, Edge, Firefox, Safari).

Ton wird über die Web Audio API erzeugt und startet erst nach der ersten Nutzer-Interaktion (Browser-Vorgabe).

3. Projektstruktur

Alle Dateien liegen flach in einem Ordner (dem GitHub-Pages-Wurzelverzeichnis):

Datei	Rolle
<code>index.html</code>	Startmenü: Logo, Level-Auswahl (3 Karten), Anleitung, Credits, Sterne-Anzeige. Einstiegspunkt der Seite.
<code>game.js</code>	Menü-Logik für <code>index.html</code> : Sound, Navigation zwischen den Menü-Screens, schwebende Icons, Sterne-Anzeige, Fortschritt laden. Enthält außerdem die Quiz-Engine, die von Level 1 genutzt wird.
<code>style.css</code>	Gemeinsames Design für Menü und Level 1 (Farben, Karten, Buttons, Quiz-Layout, Overlays).
<code>level1.html</code>	Level 1 - „Die Grundlagen“ (Quiz, eigenständige Datei, nutzt <code>style.css</code>).
<code>level2.html</code>	Level 2 - „CPU Kommandant“ (3D-Scheduling, eigenständig, Three.js).
<code>level3.html</code>	Level 3 - „PC-Baumeister“ (3D-Montage, eigenständig, Three.js).
<code>hardware-heroes-evaluation.html</code>	Separate Evaluations-/Auswertungsseite (über den Button „Evaluation“ im Menü erreichbar).
<code>three.min.js</code>	Lokale Kopie von Three.js. Hinweis: Aktuell nicht eingebunden – die Level laden Three.js über das CDN. Die Datei kann als Offline-Fallback dienen (siehe Abschnitt 9).
<code>CNAME</code>	Enthält die Domain <code>luisgetlost.me</code> für GitHub Pages.
<code>README.md</code>	Diese Dokumentation.

4. Spielaufbau: die drei Level

Level	Datei	Titel	Schwierigkeit	Aufgabentypen
1	<code>level1.html</code>	Die Grundlagen	Einfach	Multiple-Choice-Quiz + Bauteil erkennen
2	<code>level2.html</code>	CPU Kommandant	Mittel	Quiz + 3D-Scheduling in 3 Runden
3	<code>level3.html</code>	PC-Baumeister	Profi	Quiz + 3D-Montage + Kabel stecken

Level 1 - Die Grundlagen (`level1.html`)

Fünf feste Fragen zu CPU, RAM, SSD vs. HDD, GPU (Bauteil erkennen) und Mainboard. Zu jeder Frage gibt es eine Erklärung und einen Button „Mehr erfahren“ mit Detail-Infos zum Bauteil (Steckbrief mit Fakten und SVG-Grafik).

- **Punkte:** 20 pro richtige Frage, maximal 100.

- **Sterne:** $\geq 90\%$ → 3 Sterne, $\geq 60\%$ → 2 Sterne, sonst 1 Stern.

Level 2 - CPU Kommandant ([level2.html](#))

Zunächst ein kurzes Quiz zum Thema Scheduling (5 zufällige Fragen aus einem Pool von 7), danach die „Kommandozentrale“: In **drei Runden** müssen Prozesse in der richtigen Reihenfolge in die 3D-CPU geschickt werden.

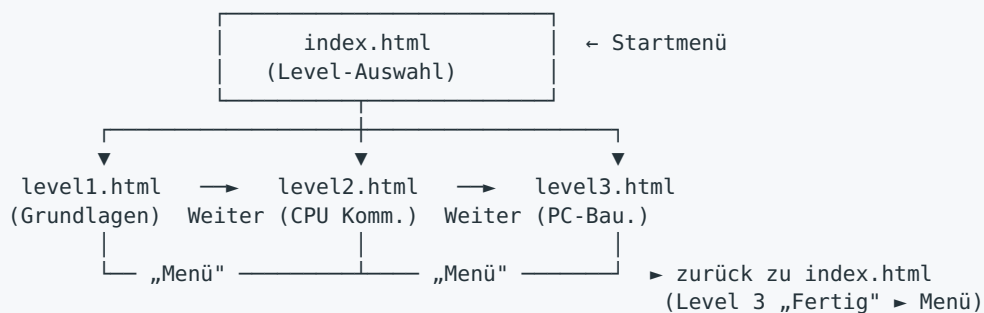
- **Runde 1 - FCFS** (First Come, First Served): nach Ankunftszeit (4 Prozesse)
- **Runde 2 - SJF** (Shortest Job First): kürzeste Rechenzeit zuerst (5 Prozesse)
- **Runde 3 - Priorität:** wichtigster Prozess zuerst, inkl. „Starvation“ (5 Prozesse)
- **Punkte:** richtige Quizfragen $\times 100$ + eingeplante Prozesse $\times 40$ + Fehlerbonus (0/100/200).
- **Sterne:** ≥ 1000 → 3, ≥ 700 → 2, sonst 1.

Level 3 - PC-Baumeister ([level3.html](#))

Quiz (5 zufällige aus 7 Fragen), danach 3D-Montage: sieben Komponenten (Mainboard, CPU, Kühler, 2 $\times$ RAM, GPU, SSD) in die richtigen Slots einbauen und drei Kabel (ATX 24-Pin, EPS 8-Pin, PCIe 8-Pin) korrekt stecken.

- **Punkte:** richtige Quizfragen $\times 100$ + 300 (Einbau) + 200 (Kabel) + Fehlerbonus.
- **Sterne:** ≥ 900 → 3, ≥ 600 → 2, sonst 1.

5. Spielfluss & Navigation



- Jede Level-Karte im Menü öffnet die jeweilige Datei.
- Am Ende jedes Levels führt **„Weiter“** zum nächsten Level, **„Menü“** zurück ins Startmenü.
- Level 3 ist das letzte Level; sein Abschluss-Button führt zurück ins Menü.

6. Technik

- **Reines Frontend:** HTML5, CSS3 und Vanilla JavaScript – keine Frameworks, kein Build.
- **3D (Level 2 & 3):** `Three.js` r128, geladen per CDN (`cdnjs.cloudflare.com/.../three.min.js`).
- **Ton:** Web Audio API (Klänge werden generativ im Code erzeugt, keine Audiodateien).
- **Speicherung:** `localStorage` im Browser (kein Server, keine Datenbank).
- **Architektur:** Menü (`index.html` + `game.js` + `style.css`) und Level 1 teilen sich Design und Quiz-Engine. Level 2 und 3 sind jeweils **eigenständige, in sich geschlossene HTML-Dateien** mit eigenem CSS und JavaScript.

7. Fortschritt & Sterne-System

Der Fortschritt wird im Browser unter dem Schlüssel `hw_stars` im `localStorage` gespeichert – als JSON-Objekt der Form `{"1":3,"2":2,"3":1}` (Level → beste Sternzahl).

- Jedes Level schreibt beim Abschluss seine beste Sternzahl dorthin.
- Das Startmenü liest diesen Wert und zeigt Gesamt-Sterne (max. **9**) und Anzahl abgeschlossener Level (max. **3**) an.

Fortschritt zurücksetzen: In der Browser-Konsole (F12)

`localStorage.removeItem('hw_stars')` ausführen und die Seite neu laden.

8. Inhalte anpassen (für Dozenten)

Alle Inhalte stehen als übersichtliche Datenstrukturen (Arrays) direkt im Quelltext. Zum Bearbeiten die jeweilige Datei in einem Texteditor öffnen.

Level-1-Fragen ändern → `level1.html`

Im `<script>`-Block gibt es das Array `const QUESTIONS = [ ... ]`. Jede Frage sieht so aus:

```
{
  type: 'quiz',           // 'quiz' = normale Frage, 'identify' = Bauteil erkennen
  part: 'cpu',            // zeigt die passende Bauteil-Grafik
  text: 'Was ist die Hauptaufgabe der CPU?',
  answers: ['...', '...', '...', '...'],
  correct: 2,              // Index der richtigen Antwort (0 = erste)
  explanation: 'Erklärung, die nach dem Antworten erscheint.'
}
```

Eine neue Frage hinzufügen: einfach einen weiteren solchen Block (durch Komma getrennt) ins Array kopieren. Die Fragenanzahl passt sich automatisch an.

Level-2-Inhalte ändern → `level2.html`

- **Quizfragen:** Array `const QUIZ = [ ... ]` (Format: `q`, `a` mit `t/c`, `fb`, `wb`). Es werden pro Durchgang **5 zufällige** Fragen aus dem Pool gezogen.
- **Scheduling-Runden:** Array `const ROUNDS = [ ... ]` (FCFS, SJF, Priorität; jeweils mit den Prozessen `procs` inkl. Ankunftszeit `arr`, Rechenzeit `burst`, Priorität `prio`).

Level-3-Inhalte ändern → `level3.html`

- **Quizfragen:** Array `const QUIZ = [ ... ]` (ebenfalls 5 von 7).
- **Komponenten & Slots:** Arrays `const COMPS` und `const SLOTS`.
- **Kabel:** Array `const CABLES`.

Sterne-Schwellen ändern

Die Grenzen stehen jeweils in der Funktion `showWin()` bzw. `showResult()`:

- Level 1 (`level1.html`): `let starCount = pct >= 0.9 ? 3 : pct >= 0.6 ? 2 : 1;`
- Level 2 (`level2.html`): `const stars = pts>=1000?3:pts>=700?2:1;`
- Level 3 (`level3.html`): `const stars = pts>=900?3:pts>=600?2:1;`

9. Wartung & Hinweise

- **Neue Level oder Umbenennungen:** Die Menü-Verlinkung steht in `index.html` in den drei `level-card`-Elementen (`window.location.href='levelX.html'`). Die „Weiter“- und „Menü“-Ziele stehen in den Level-Dateien in den Funktionen `finishLevel()` und `goBack()` (Level 2 & 3) bzw. in den Button-`onclick`-Attributen (Level 1).
- **Offline-Betrieb der 3D-Level:** Aktuell wird Three.js vom CDN geladen. Für den Offline-Einsatz in `level2.html` und `level3.html` die Zeile `<script src="https://cdnjs.cloudflare.com/.../three.min.js"></script>` durch `<script src="three.min.js"></script>` ersetzen (die lokale Datei liegt bereits im Ordner).
- **Browser:** Empfohlen wird ein aktueller Desktop-Browser. Die 3D-Level benötigen WebGL.

10. Deployment (GitHub Pages)

Das Projekt wird über **GitHub Pages** aus dem Repository `lirrix.github.io` veröffentlicht und ist über die Domain **luisgetlost.me** (Datei `CNAME`) erreichbar.

Änderungen veröffentlichen: Die geänderten Dateien in das Repository übertragen (committen und pushen). GitHub Pages aktualisiert die Live-Seite anschließend automatisch (meist innerhalb einer Minute).

11. Credits

Entwickelt von **Luis, Genta, Angelika und Elyesa**. Technologien: HTML5 · CSS3 · Vanilla JavaScript · Three.js · Web Audio API.